

[Home](#)[GitHub](#)

## CLASSES

[ArticlesService](#)[ErrorBoundary](#)

## EVENTS

[SIGINT](#)[unhandledRejection](#)

## NAMESPACES

[apiConfig](#)[contactoEmergenciaService](#)[diarioService](#)

## GLOBAL

[404\\_Error\\_Handler](#)[ACTIVIDADES](#)[API\\_URL](#)[ASPECTOS\\_COGNITIVOS](#)[ActionCard](#)[ActivitySelector](#)[App](#)[Articulos](#)[AuthButtons](#)[AuthLayout](#)[Button](#)[CORS\\_Configuration](#)[Card](#)[Carousel](#)[Checkbox](#)[CircularProgress](#)[CognitionSelector](#)[Diario](#)[DiaryBanner](#)[DiaryEditor](#)[DiaryEntry](#)[EMOCIONES](#)[EmergencyButton](#)[EmergencyModal](#)[EmotionSelector](#)[EmotionStats](#)

# backend/src/server.js

```
1.  /**
2.   * @file Fichero de arranque del servidor.
3.   * @description Conecta a la base de datos MongoDB y levanta el serv
4.   * @requires mongoose
5.   * @requires dotenv
6.   * @requires ./app
7.   */
8.  const mongoose = require('mongoose');
9.  require('dotenv').config();
10.
11. // Importar la aplicación Express configurada
12. const app = require('./app');
13.
14. /**
15.  * @function validateRequiredEnvVars
16.  * @description Valida que las variables de entorno críticas estén c
17.  */
18. function validateRequiredEnvVars() {
19.   const requiredVars = [
20.     'MONGODB_URI',
21.     'JWT_SECRET',
22.   ];
23.
24.   const missingVars = requiredVars.filter(varName => {
25.     // Comprobar alias
26.     if (varName === 'MONGODB_URI' && (process.env.MONGODB_URI ||
27.       return false;
28.     }
29.     return !process.env[varName];
30.   });
31.
32.   if (missingVars.length > 0) {
33.     console.error('  ERROR: Variables de entorno requeridas no
34.     missingVars.forEach(varName => {
35.       console.error(`    - ${varName}`);
36.     });
37.     console.error('\n  Crea un archivo .env basado en .env.exam
38.     console.error('    0 ejecuta: ./setup-env.sh para generar aut
39.     process.exit(1);
40.   }
41.
42.   // Validar longitud mínima del JWT_SECRET
43.   if (process.env.JWT_SECRET && process.env.JWT_SECRET.length < 32
44.     console.error('  ERROR: JWT_SECRET debe tener al menos 32 c
45.     console.error('  Genera uno nuevo con: openssl rand -base64
46.     process.exit(1);
47.   }
48.
49.   console.log('  Variables de entorno validadas correctamente');
50. }
51.
52. // Validar variables de entorno antes de continuar
53. validateRequiredEnvVars();
54.
```

```

55.
56. /**
57.  * @constant {number} port
58.  * @description El puerto en el que se ejecutará el servidor.
59.  */
60. // --- Configuración del Puerto ---
61. const port = process.env.PORT || process.env.PUERTO_BACKEND || 3000;
62.
63. /**
64.  * @constant {string} uri
65.  * @description La URI de conexión a la base de datos MongoDB.
66.  */
67. // --- Configuración de MongoDB con Mongoose ---
68. const uri = process.env.MONGODB_URI || process.env.URL_DB;
69. if (!uri) {
70.     console.error("La variable de entorno MONGODB_URI o URL_DB no es");
71.     process.exit(1);
72. }
73.
74. /**
75.  * @function connectDBAndStartServer
76.  * @description Conecta a la base de datos MongoDB y, si tiene éxito
77.  * @async
78.  */
79. async function connectDBAndStartServer() {
80.     try {
81.         // Conectar a MongoDB usando Mongoose
82.         await mongoose.connect(uri);
83.         console.log("    Conectado exitosamente a MongoDB con Mongoose");
84.
85.         // Iniciar el servidor Express para que escuche peticiones
86.         app.listen(port, () => {
87.             console.log(`    Servidor corriendo en http://localhost:${port}`);
88.             console.log(`    Health check: http://localhost:${port}/api/health`);
89.             console.log(`    Auth API: http://localhost:${port}/api/auth`);
90.         });
91.
92.     } catch (error) {
93.         // Manejo de errores de conexión a la base de datos
94.         console.error("    No se pudo conectar a la base de datos:", error);
95.         process.exit(1);
96.     }
97. }
98.
99. // Llamar a la función para conectar a la BD e iniciar el servidor
100. connectDBAndStartServer();
101.
102. /**
103.  * @event SIGINT
104.  * @description Maneja el cierre de la aplicación (Ctrl+C) para cerrar
105.  */
106. // Manejar el cierre de la aplicación para cerrar la conexión a la BD
107. process.on('SIGINT', async () => {
108.     console.log('\n    Cerrando la conexión a la base de datos...');
109.     await mongoose.connection.close();
110.     console.log('    Conexión cerrada. Adiós!');
111.     process.exit(0);
112. });
113.
114. /**
115.  * @event unhandledRejection
116.  * @description Maneja promesas rechazadas no capturadas para evitar
117.  */
118. // Manejar errores no capturados

```

```
119.   process.on('unhandledRejection', (err) => {
120.       console.error('  Unhandled Rejection:', err);
121.       mongoose.connection.close();
122.       process.exit(1);
123.   });
```

Documentation generated by [JSDoc 4.0.5](#) on Thu Dec 11 2025 19:37:23 GMT+0000  
(Coordinated Universal Time) using the [docdash](#) theme.